

南开大学

Nankai University

南　　开　　大　　学

计　算　机　学　院

编译原理上机实验报告

预备工作：了解你的编译器、LLVM IR 与汇编编程

姓名：张龙飞、袁煜杰

学号：2413625、2412303

专业：计算机科学与技术

2026 年 10 月 7 日

摘要

【待补全：摘要正文】

关键字：编译器；LLVM IR；RISC-V 汇编；SysY；链接

目录

一、 引言 **1**

（一） 实验背景与目的 1

（二） 实验环境（张龙飞） 1

（三） 实验环境（袁煜杰） 1

（四） 小组分工 1

二、 编译系统工作流程研究（张龙飞） **2**

（一） 实验方案 2

（二） 预处理器 2

（三） 编译器 3

1. 编译前端：生成 LLVM IR 3

2. 编译后端：生成目标汇编 4

（四） 汇编器 4

（五） 链接器 5

（六） 运行验证 5

（七） 进一步探索：优化选项的影响 5

三、 编译系统工作流程研究（袁煜杰） **6**

四、 LLVM IR 与汇编编程实践 **6**

（一） SysY 示例程序设计 6

（二） 手写 LLVM IR 程序及验证 6

（三） 手写 RISC-V 汇编程序及验证 7

（四） 等价性验证与结果分析 8

五、 总结与体会 **8**

一、 引言

(一) 实验背景与目的

SysY 是全国大学生编译系统设计赛定义的简化 C 语言。本课程上机大作业要求实现一个 SysY 编译器（词法分析、语法分析、类型检查与中间代码生成、代码优化、目标代码生成），目标平台为 RISC-V。本次预备工作是上机大作业的第一个模块，目的在于：

1. 以 GCC / LLVM(Clang) 为研究对象，深入探究语言处理系统的完整工作过程，理解预处理器、编译器（前端与后端各阶段）、汇编器、链接器的功能；
2. 熟悉 LLVM IR 中间语言与 RISC-V 汇编语言，设计覆盖 SysY 语言特性的示例程序，手写等价的 LLVM IR 与 RISC-V 汇编程序，链接 SysY 运行时库并验证运行结果的正确性；
3. 为后续词法分析器、语法分析器、中间代码生成与目标代码生成等模块的实现建立直观认识与技术储备。

(二) 实验环境（张龙飞）

本实验在 WSL2（Windows Subsystem for Linux）中进行，使用 QEMU 用户态模拟器运行 RISC-V 目标程序。主要工具及版本如表 1 所示。

项目	版本 / 说明
操作系统	WSL2：Ubuntu 22.04.5 LTS（x86_64）
GCC / G++	11.4.0
Clang / LLVM 工具链	14.0.0（含 llvm-as、lli、llc、opt、llvm-dis）
Flex / Bison	2.6.4 / 3.8.2
RISC-V 交叉工具链	riscv64-linux-gnu-gcc / g++ 11.4.0
模拟器	QEMU 7.0.0（qemu-riscv64 用户态模式，/opt/qemu 源码编译）

表 1: 实验环境与工具版本

(三) 实验环境（袁煜杰）

- 待填写

(四) 小组分工

本实验由两人合作完成，分工如表 2 所示。

成员	学号	分工	独立撰写部分
【成员 A 姓名】	【学号】	小题二任务一：LLVM IR编程	小题一（独立完成）
【成员 B 姓名】	【学号】	小题二任务二：RISC-V 汇编编程	小题一（独立完成）

表 2: 小组成员及分工

报告整体框架由两人共同构建；摘要、引言、结论等部分共同撰写；小题一由两人各自独立完成实验并独立撰写；小题二按分工分别撰写各自负责的任务，实验结果合并展示。

NKU

二、 编译系统工作流程研究（张龙飞）

本小题以 PPT 示例的 C++ 版阶乘程序 `fact.cpp` 为研究载体，利用编译器的命令行选项逐步获得各阶段输出（实验方案见表 3），研究其与源程序的关系。

(一) 实验方案

研究载体：阶乘程序 fact.cpp（PPT 示例）

```cpp
#include <iostream>
using namespace std;
int main()
{
    int i, n, f;

    cin >> n;
    i = 2;
    f = 1;
    while (i <= n)
    {
        f = f * i;
        i = i + 1;
    }
    cout << f << endl;
}
```

阶段	命令选项	产物
预处理	`clang++ -E`	`fact.i`
编译（前端）	`clang++ -emit-llvm -S`	`fact.ll`（LLVM IR）
编译（后端）	`g++ -S` / `riscv64-linux-gnu-g++ -S`	`fact.s`（汇编）
汇编	`clang++ -c`	`fact.o`（目标文件）
链接	`g++` / `riscv64-linux-gnu-g++ -static`	可执行程序
运行验证	`lli` / `qemu-riscv64`	运行结果

表 3: 各编译阶段对应的命令选项与产物

(二) 预处理器

源程序仅 17 行，经 `clang++ -E` 预处理后得到 30194 行的 `fact.i`——C++ 标准库头文件 `<iostream>` 展开了数万行声明。

NKU

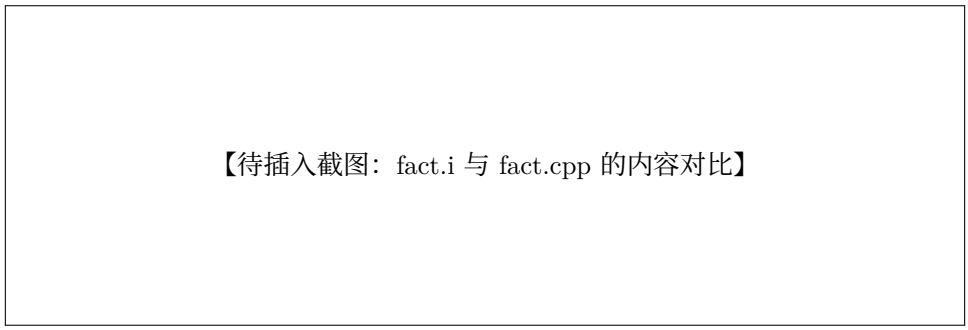


图 1: 预处理输出与源程序的对比

【待补全：预处理前后对比分析，例如 `#include` 与 `#define` 的处理、注释的去留、可用 `-D` 选项自定义宏做进一步观察，以及 C++ 头文件展开规模远大于 C 头文件的原因】

(三) 编译器

1. 编译前端：生成 LLVM IR

`clang++ -emit-llvm -S` 生成 111 行 LLVM IR（`fact.ll`），关键片段如下：

fact.ll 关键片段

```
@_ZSt3cin  = external global %"class.std::basic_istream", align 8
@_ZSt4cout = external global %"class.std::basic_ostream", align 8

define dso_local noundef i32 @main() #4 {
  %1 = alloca i32, align 4        ; return value
  %2 = alloca i32, align 4        ; i
  %3 = alloca i32, align 4        ; n
  %4 = alloca i32, align 4        ; f
  store i32 0, i32* %1, align 4
  %5 = call ... @_ZNSirsERi(@_ZSt3cin, i32* %3)      ; cin >> n
  store i32 2, i32* %2, align 4                      ; i = 2
  store i32 1, i32* %4, align 4                      ; f = 1
  br label %6
6:                                                   ; while (i <= n)
  %7 = load i32, i32* %2, align 4
  %8 = load i32, i32* %3, align 4
  %9 = icmp sle i32 %7, %8
  br i1 %9, label %10, label %16
10:
  %11 = load i32, i32* %4
  %12 = load i32, i32* %2
  %13 = mul nsw i32 %11, %12                         ; f = f * i
  ...
16:
  %17 = load i32, i32* %4, align 4
  %18 = call ... @_ZNSolsEi(@_ZSt4cout, i32 %17)     ; cout << f
  %19 = call ... @_ZNSolsEPFRSoS_E(%18, ... endl)    ; << endl
```

NKU

```
  ret i32 %20
}
```

【待插入：fact.ll 控制流图（已生成：prep/stages/fact/fact_main_cfg.png）】

图 2: LLVM IR 与控制流结构

【待补全：IR 与源程序的对应关系分析】

2. 编译后端：生成目标汇编

分别用 `g++ -S`（x86_64，AT&T 风格）与 `riscv64-linux-gnu-g++ -S`（RISC-V）生成汇编，对比两者风格。

【待插入截图：x86 汇编与 RISC-V 汇编对比】

图 3: 同一代码在 x86 与 RISC-V 下的汇编对比

【待补全：x86 与 RISC-V 汇编的对比分析】

（四） 汇编器

`clang++ -c` 将汇编翻译为目标文件 `fact.o`，可用 `objdump -d` 反汇编查看机器码，`objdump -r` 查看重定位项。

【待插入截图：objdump -d / objdump -r 输出】

图 4: 目标文件的反汇编与重定位信息

NKU

【待补全：汇编器功能分析】

（五） 链接器

链接阶段将目标文件与 C++ 运行库（`std::cin`/`std::cout` 所在的 libstdc++）组装为可执行程序。动态链接与静态链接产物对比如图 5 所示。

【待插入截图：ldd 输出 / 动态与静态可执行文件大小对比】

图 5: 动态链接与静态链接对比

【待补全：链接器功能分析】

（六） 运行验证

四种方式运行同一程序，结果一致（输入 5，输出 120），说明各阶段产物等价：

运行验证

```
$ echo 5 | ./fact_x86                # x86 可执行（g++ 链接产物）
120
$ echo 5 | lli fact.ll               # LLVM IR 解释执行
120
$ echo 5 | ./fact_fromll_x86         # 由 IR 直接重编译的可执行
120
$ echo 5 | qemu-riscv64 ./fact_rv64  # RISC-V 可执行（QEMU 模拟）
120
```

（七） 进一步探索：优化选项的影响

调整优化级别（`-O0/-O1/-O2/-Os`），观察 LLVM IR 与 RISC-V 汇编的变化，结果如表 4 所示。

程序 / 产物	-O0	-O1	-O2	-Os
fact：LLVM IR 行数	111	163	250	152
fact：RISC-V 汇编行数	156	97	146	89
fib：LLVM IR 行数	124	248	247	153
fib：RISC-V 汇编行数	177	156	231	118

表 4: 不同优化级别下 LLVM IR 与 RISC-V 汇编的规模对比

【待补全：结合表 4 及更多探索（如 `-g` 选项、微调源程序）分析优化器行为】

三、 编译系统工作流程研究（袁煜杰）

- 待补全

四、 LLVM IR 与汇编编程实践

(一) SysY 示例程序设计

围绕上机大作业需要支持的 SysY 特性设计示例程序，特性覆盖情况如表 5 所示。

特性类别	覆盖点	示例程序
数值运算	+、-、*、/、%，单目 +/-，关系与逻辑运算	【如 calc.sy】
语句	赋值、语句块、if/else、while、break/continue、return	【如 branch_loop.sy】
函数	声明、调用、参数传递、递归	【fact 递归版等】
常量与变量	const、初始化、嵌套作用域	【如 scope.sy】
运行时库	getint/putint/putch/getch 交互	【各示例均使用】
进阶（选做）	数组、浮点数	【如 array.sy】

表 5: SysY 特性覆盖设计

本报告以循环版阶乘程序（SysY 风格，通过 getint/putint 与运行时库交互）作为入门示例，完整覆盖各特性的示例程序见附件。

(二) 手写 LLVM IR 程序及验证

手写 LLVM IR 采用含 ϕ 节点的标准 SSA 形式，与 SysY 源程序语义等价：

手写 LLVM IR：fact_manual.ll

```
declare i32 @getint()
declare void @putint(i32)
declare void @putch(i32)

define i32 @main() {
entry:
  %n = call i32 @getint()
  br label %loop
loop:
  %i = phi i32 [ 2, %entry ], [ %i_next, %body ]
  %f = phi i32 [ 1, %entry ], [ %f_next, %body ]
  %cond = icmp sle i32 %i, %n
  br i1 %cond, label %body, label %done
body:
  %f_next = mul nsw i32 %f, %i
  %i_next = add nsw i32 %i, 1
  br label %loop
done:
  call void @putint(i32 %f)
```

```
  call void @putch(i32 10)
  ret i32 0
}
```

链接 SysY 运行时库（`sylib.c`）编译并运行，输入 7 输出 5040，结果正确：

```
$ clang fact_manual.ll sylib.c -o fact_manual_ll
$ echo 7 | ./fact_manual_ll
5040
```

进一步地，手写 IR 可经 `llc` 自动下降到 RISC-V 汇编并运行，体现 LLVM IR 作为跨平台中间层的作用：

```
$ llc -march=riscv64 -mattr=+m,+a,+f,+d,+c fact_manual.ll -o fact_rv.s
$ riscv64-linux-gnu-gcc -static fact_rv.s sylib.c -o fact_fromll_rv
$ echo 7 | qemu-riscv64 ./fact_fromll_rv
5040
```

【待补全：更多示例程序的手写 IR 与验证过程】

(三) 手写 RISC-V 汇编程序及验证

手写 RISC-V（RV64）汇编遵循标准调用约定（a0–a7 传参、ra 保存返回地址、s0–s11 为被调者保存寄存器）：

手写 RISC-V 汇编：fact_manual_rv.s（关键片段）

```
main:
    addi  sp, sp, -32
    sd    ra, 24(sp)
    sd    s0, 16(sp)
    sd    s1, 8(sp)
    sd    s2, 0(sp)
    call  getint          # a0 = n
    mv    s0, a0
    li    s1, 2           # i = 2
    li    s2, 1           # f = 1
.Lloop:
    bgt   s1, s0, .Ldone  # while (i <= n)
    mul   s2, s2, s1      # f = f * i
    addi  s1, s1, 1       # i = i + 1
    j     .Lloop
.Ldone:
    mv    a0, s2
    call  putint
    li    a0, 10
    call  putch
    li    a0, 0
    ld    ra, 24(sp)
    ld    s0, 16(sp)
    ld    s1, 8(sp)
```

NKU

```
    ld      s2, 0(sp)
    addi    sp, sp, 32
    ret
```

交叉编译并通过 QEMU 运行，输入 7 输出 5040，结果正确：

```
$ riscv64-linux-gnu-gcc -static fact_manual_rv.s sylib.c -o fact_manual_rv
$ echo 7 | qemu-riscv64 ./fact_manual_rv
5040
```

【待补全：更多示例程序的手写汇编与验证过程，可对比手写汇编与 `riscv64-linux-gnu-gcc -S` 自动生成汇编的差异】

（四） 等价性验证与结果分析

所有产物对同一输入的运行结果一致，验证了手写 IR / 手写汇编与源程序的等价性，汇总如表 6 所示。

示例程序	特性	手写 IR 运行	手写汇编运行	预期结果
fact（$n = 7$）	循环/运算/库调用	5040 ✓	5040 ✓	5040
【待补充】				
【待补充】				

表 6: 示例程序的等价性验证结果

【待补全：结果分析与体会】

五、 总结与体会

【待补全：总结与体会】

NKU